

Final Report-*Unified Generative AI Models via Command-Line Interface*

Araf Tahsan Pavel, ID: 2212079042

Md Tanveer Hossain Nihal, ID: 2211935042

Ismot Ara Emu, ID: 2212108042

*Department of Electrical and Computer Engineering
North South University*

Section: 05

Group: 06

Abstract—This project integrates multiple generative models (GPT-2, DCGAN, CGAN, CVAE, WaveGAN) into a unified CLI tool that allows training, evaluation, and generation using a structured directory and file-based workflow. It aims to streamline experimentation in generative modeling by reducing the overhead of managing different frameworks and datasets separately. The tool supports text and audio modalities with model-specific training and sampling routines.

Index Terms—Deep Learning, GPT-2, GANs, CVAE, CLI, Text Generation, Audio Synthesis, Multimodal Learning

I. INTRODUCTION

A. Background and Motivation

In recent years, deep learning has revolutionized the field of artificial intelligence by enabling machines to generate human-like content across multiple domains such as text, images, and audio. With the growing demand for multi-domain generative models, researchers have developed specialized architectures such as Generative Adversarial Networks (GANs), Conditional Variational Autoencoders (CVAE), and autoregressive language models like GPT-2. Despite their individual strengths, integrating these models into a single, unified pipeline remains a challenge due to variations in data formats, training procedures, and model objectives.

This project presents a unified command-line interface (CLI) that seamlessly incorporates multiple generative deep learning models—namely GPT-2, DCGAN, CGAN, CVAE, and WaveGAN—under a standardized architecture. The goal is to provide a modular and extensible framework where researchers and developers can train, evaluate, and generate outputs using different types of generative models from a single interface. This design promotes reproducibility, standardization, and accessibility, which are essential for rapid prototyping and experimentation in generative AI.

The system also addresses complex engineering problems such as managing interdependencies between models, ensuring consistent dataset loading procedures, and dynamically allocating resources for GPU-based training. Each model follows its own optimized training strategy, while the CLI harmonizes these approaches through a common interface and consistent result logging.

This unified CLI can support a variety of use cases, including text generation with fine-tuned GPT-2 models, image synthesis using GAN-based models, and waveform audio generation via WaveGAN. Furthermore, the modularity of the codebase allows easy extension to future models and datasets.

By solving challenges across data handling, model diversity, and user interface design, this project contributes toward the development of robust and reusable tools for generative AI research. It also fulfills key attributes of Complex Engineering Problems (CEP), particularly in areas such as depth of knowledge (P1), model interdependence (P7), and managing conflicting requirements (P2).

B. Purpose and Goal of the Project

The primary objective of this project is to design and implement a unified, modular command-line interface (CLI) that allows users to train, evaluate, and generate outputs from multiple deep generative models across different data domains. These include text, image, and audio modalities, integrated through a standard execution pipeline.

The project aims to simplify the use and experimentation of diverse generative architectures—including GPT-2 for text generation, DCGAN and CGAN for image generation, CVAE for conditional image generation, and WaveGAN for raw audio synthesis—by providing a single entry point via the command:

```
python ../codeP/main.py --dataset  
datasetName --model modelName  
--otherOptions otherOptions
```

To achieve this, the project defines the following specific goals:

- **Unified Execution:** Enable all models to be launched and managed through a consistent CLI pattern, regardless of modality (text/image/audio) or architecture type (GAN, VAE, Transformer).
- **Modular Design:** Organize the project into separate components for models (`codeP/models`), datasets (`codeP/datasets`), bash wrappers (`codeB`), and output handling (`RESULTS/`), to ensure reusability and ease of extension.

- **Standardized Dataset Handling:** Ensure consistent data preprocessing and loading procedures by placing training and testing datasets in a common directory format:

DATA/datasetName/training/ and
DATA/datasetName/testing/

- **Result Management:** Store all training logs, model checkpoints, and generated outputs (e.g., text files, image samples, audio waveforms) in a centralized folder structure under `RESULTS/`, promoting organization and experiment tracking.
- **Cross-Modality Support:** Facilitate generative modeling in different domains—language, vision, and sound—by adapting each model’s input/output pipeline to a shared interface structure.
- **Reproducibility:** Provide shell scripts (`.sh` files in `codeB/`) that encapsulate typical training and inference workflows, making the entire pipeline reproducible for different users.
- **Scalability:** Allow integration of future generative models (e.g., diffusion models or transformers) by adhering to the modular template and CLI standard defined in this framework.
- **Educational Value:** Serve as a reference project for students and researchers to understand and compare the behavior of different generative AI models under a common environment.

By fulfilling these goals, the project aims to contribute a robust and user-friendly framework that enhances the usability and interpretability of complex deep learning models in generative tasks.

C. Report Organization

This report is structured to provide a comprehensive overview of the development and implementation of a unified command-line interface (CLI) for multiple generative deep learning models. Chapter 1 introduces the motivation, scope, and goals of the project. Chapter 2 presents a review of existing literature related to generative models such as GANs, VAEs, and autoregressive transformers, along with their respective limitations and research gaps.

Chapter 3 outlines the system architecture, including the modular file structure and integration logic that supports seamless switching between models and datasets. Chapter 4 details the methodology, covering dataset preparation, training workflows, evaluation procedures, and CLI execution logic.

Chapter 5 analyzes the impacts of the project on software development practices, reproducibility, and educational usability. Chapter 6 discusses the project planning and budgeting aspects. Chapter 7 focuses on the mapping of this work to Complex Engineering Problems (CEPs) and Complex Engineering Activities (CEAs), with emphasis on P1 (Depth of Knowledge), P3 (Depth of Analysis), and A1–A3 activities. Chapter 8 concludes the report with a summary, identified limitations, and suggestions for future enhancements.

II. LITERATURE REVIEW

A. Related Work

Generative deep learning models have achieved remarkable progress in recent years across various domains such as text generation, image synthesis, and audio generation. Notable among these are transformer-based models like GPT-2, which have demonstrated state-of-the-art performance in natural language processing through large-scale unsupervised pre-training and fine-tuning [?]. GPT-2 fine-tuning frameworks such as Hugging Face’s Transformers library offer robust pipelines, but are often coupled with heavy computational demands and require manual adjustments for new datasets or configurations.

In the domain of image generation, GAN architectures such as DCGAN [?], CGAN [?], and WaveGAN [?] have enabled conditional and unsupervised generation of visual and audio content. While these models achieve impressive results, they are frequently implemented and deployed as isolated scripts with dataset-specific code, which limits reusability and hinders experimentation across modalities. Similarly, Conditional Variational Autoencoders (CVAEs) have shown effectiveness in learning interpretable latent representations, but their integration into unified frameworks remains limited.

Existing research often treats these generative models independently, resulting in fragmented workflows that are difficult to replicate or scale. Moreover, the high computational cost of training such models individually poses significant challenges, particularly in environments with limited hardware resources. These constraints motivated the development of a unified CLI-based framework in this project, which streamlines execution and minimizes overhead by abstracting model training and evaluation behind a single interface.

By consolidating diverse generative model pipelines into a shared infrastructure, this work addresses the practical challenges of resource-constrained environments and supports educational and experimental use cases.

III. SYSTEM METHODOLOGY

A. Architecture Overview

The unified generative AI CLI framework is designed with a modular architecture to support seamless integration, training, and sampling from various generative models including GPT-2, GPT-Neo, DCGAN, CGAN, CVAE, and WaveGAN. This modularity ensures extensibility, maintainability, and consistency across all model pipelines, and is especially beneficial for resource-constrained setups and reproducible research.

The architecture consists of the following core modules:

- **Model Loader:** Dynamically imports and initializes the selected model based on the `--model` argument. This component abstracts architecture-specific logic and ensures a standard training and sampling interface across models.
- **Dataset Parser:** Loads and preprocesses datasets according to the model type (text, image, or audio). This

includes transformations, batching, and normalization for consistent model input.

- **Trainer Module:** Orchestrates training by configuring optimizers, computing losses, and logging performance. For GANs, both generator and discriminator are updated. For text models, tokenization and sequence generation are handled.
- **Sampler/Inference Engine:** Performs inference and generates final outputs. It supports sampling from trained models and saving results in formats such as .png, .wav, or .txt under the RESULTS/ directory.
- **Command-Line Interface (CLI):** A lightweight wrapper script (main.py) using argparse enables users to specify models and datasets via terminal. This streamlines reproducibility and allows automation through shell scripts located in codeB/.

CLI Example:

```
python ../codeP/main.py --dataset datasetName --model
modelName --otherOptions optionName
```

Main Driver Snippet:

```
# main.py
import argparse
from runner import run_pipeline

parser = argparse.ArgumentParser()
parser.add_argument('--dataset', required=True)
parser.add_argument('--model', required=True)
parser.add_argument('--otherOptions', required=False,
                    default=None)

args = parser.parse_args()

if __name__ == "__main__":
    run_pipeline(args.model, args.dataset, args.otherOptions)
```

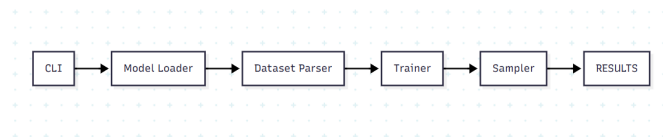


Fig. 1: Modular Pipeline of the Unified CLI Framework for Generative Models

B. Model Support

The unified CLI framework supports multiple generative models across text, image, and audio domains, providing a versatile platform for various applications. The following models are included in the framework:

- **GPT-2 for Text Generation:** GPT-2 is a transformer-based model developed by OpenAI for generating coherent and contextually relevant text. This model is fine-tuned on custom datasets using the Hugging Face `transformers` library and can generate human-like text based on an initial input prompt. GPT-2 is ideal for tasks such as text completion, storytelling, and dialogue generation.

- **DCGAN, CGAN, CVAE for Image Synthesis:** These models are designed for generating images from random noise vectors.

- **DCGAN (Deep Convolutional GAN):** A GAN architecture tailored for generating high-quality images, especially suited for unsupervised learning tasks. It utilizes convolutional layers in both the generator and discriminator, which allows it to produce realistic images.
- **CGAN (Conditional GAN):** This extension of the GAN model generates images conditioned on additional information, such as labels or categories. It is particularly useful when the goal is to generate images belonging to a specific class, such as generating images of cats or dogs from the CIFAR-10 dataset.
- **CVAE (Conditional Variational Autoencoder):** Unlike GANs, CVAEs generate images by learning a probabilistic distribution over the latent space. This model is effective at learning structured representations and generating class-conditional images.

- **WaveGAN for Audio Synthesis:** WaveGAN is a generative adversarial network (GAN) specifically designed for raw audio waveform generation. Unlike traditional audio generation models that rely on spectrograms, WaveGAN operates directly on waveform data. It is suitable for tasks such as music generation, speech synthesis, and environmental sound generation. WaveGAN generates high-quality audio samples from random noise vectors by leveraging 1D convolutions.

Each of these models is implemented as a modular component within the CLI framework, allowing users to easily switch between models and datasets with minimal configuration. The unified interface ensures seamless execution and facilitates cross-modality experimentation.

C. Dataset Pipeline

1) **Text Dataset Pipeline for GPT-2:** The dataset for GPT-2 is processed from multiple file formats (TXT, DOCX, PDF) using a modular approach. The steps include:

- **Data Ingestion:** Read text files from the provided directory. The code supports various formats like .txt, .docx, and .pdf.
- **Data Cleaning:** Unwanted characters, extra whitespaces, and non-textual content are removed to ensure high-quality input data.
- **Tokenization:** Using the GPT-2 tokenizer, the raw text is tokenized into manageable chunks.
- **Block Size:** The tokenized text is divided into blocks (e.g., 256 tokens per block) to fit the GPT-2 model's input requirements.
- **Dataset Creation:** The processed text data is then converted into a dataset format compatible with Hugging Face's `TextDataset` for training.

2) **Image Dataset Pipeline for DCGAN, CGAN, and CVAE:** The dataset for image generation models (DCGAN, CGAN,

and CVAE) involves several preprocessing steps to ensure images are in a consistent format for model training:

- **Data Loading:** Images are loaded from predefined directories (e.g., `DATA/datasetName/`) and are processed as per the model's requirement.
- **Transformations:** Common transformations such as resizing, normalizing to a range of `[-1, 1]`, and data augmentation are applied to the images.
- **Batching:** The images are batched into appropriate sizes for processing in the generator and discriminator networks.
- **Conditional Labels (for CGAN):** In the case of CGAN, each image is paired with its corresponding label, which is used to condition the image generation process.
- **Dataset Creation:** The processed images are then placed into a `Dataset` class to enable batch processing during model training.

3) *Audio Dataset Pipeline for WaveGAN:* For WaveGAN, the dataset consists of raw audio waveforms that need specific preprocessing before training:

- **Data Loading:** Audio files (e.g., `.wav`, `.mp3`, `.flac`) are loaded from the audio dataset directory.
- **Resampling:** The audio files are resampled to a fixed sample rate (e.g., 16kHz) to ensure consistency across inputs.
- **Mono Conversion:** Stereo audio is converted into mono to reduce computational complexity.
- **Padding or Cropping:** Audio clips are either padded or cropped to ensure a uniform length (e.g., 16,384 samples).
- **Normalization:** Audio waveforms are normalized to the range of `[-1, 1]` to standardize input values.
- **Dataset Creation:** The processed audio waveforms are converted into a `PyTorch Dataset` object, which is then used during training.

IV. IMPLEMENTATION AND CLI WORKFLOW

A. Command Structure

The project uses a command-line interface (CLI) for training and inference. The following are examples of how to use the CLI for different models:

Usage Examples:

To run the GPT-2 model for text generation:

```
python main.py --model gpt2 --dataset text
```

To run the CGAN model for generating images from the "flowers" dataset:

```
python main.py --model cgan --dataset flowers
```

Explanation: The `--model` argument specifies which model to use (e.g., `gpt2`, `cgan`, etc.), while the `--dataset` argument specifies the dataset. The `--otherOptions` argument is optional and can be used to pass additional model-specific options.

B. Training Routines

Each model follows a structured approach for training, including data loading, model initialization, training, and evaluation. Below is an overview of how the models are trained using their respective datasets.

1) *GPT-2 Training Routine:* The GPT-2 model is fine-tuned using the Hugging Face Transformers library. It is pre-trained on a large corpus of text and can be fine-tuned on specific datasets for tasks such as text completion or generation. The model is imported from the `transformers` library, and the training is conducted using the Hugging Face Trainer API. **Training Code Snippet for GPT-2:**

```
from transformers import GPT2Tokenizer, GPT2LMHeadModel,
    Trainer, TrainingArguments

# Load pre-trained GPT-2 model and tokenizer from Hugging
    Face
model_name = 'gpt2' # Use 'gpt2-medium' or 'gpt2-large' for
    larger models
tokenizer = GPT2Tokenizer.from_pretrained(model_name)
model = GPT2LMHeadModel.from_pretrained(model_name)

# Preprocess and tokenize the dataset (assuming the dataset
    is loaded in a variable 'train_dataset')
train_dataset = load_dataset_for_training(train_file_path,
    tokenizer)

# Set up training arguments
training_args = TrainingArguments(
    output_dir='./results',
    overwrite_output_dir=True,
    num_train_epochs=3,
    per_device_train_batch_size=2,
    save_steps=10_000,
    save_total_limit=2,
    logging_steps=500,
)

# Use Hugging Face's Trainer API to train the model
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
)

# Start training
trainer.train()
```

Explanation: - The `GPT2Tokenizer` and `GPT2LMHeadModel` are imported from Hugging Face's `transformers` library. - Training parameters such as `num_train_epochs`, `per_device_train_batch_size`, and `logging_steps` are defined in the `TrainingArguments` object. - The `Trainer` class handles the training loop, including saving checkpoints and logging progress.

2) *DCGAN, CGAN, CVAE Training Routines:* For the GAN-based models (DCGAN, CGAN, and CVAE), the training routines are somewhat similar, with the difference being in how the dataset is conditioned (for CGAN and CVAE). These models are implemented using `PyTorch` for flexible customization and optimization. Below is an example of how to train a DCGAN model.

Training Code Snippet for DCGAN:

```
import torch
import torch.nn as nn
```

```

import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from models import DCGAN

# Load the dataset (e.g., CIFAR-10)
transform = transforms.Compose([
    transforms.Resize(64),
    transforms.ToTensor(),
    transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
])

dataset = datasets.CIFAR10(root='./DATA', train=True,
    download=True, transform=transform)
dataloader = DataLoader(dataset, batch_size=128, shuffle=True)

# Instantiate the DCGAN model
dcgan = DCGAN()

# Set up optimizers
optimizer_G = optim.Adam(dcgan.generator.parameters(), lr=0.0002, betas=(0.5, 0.999))
optimizer_D = optim.Adam(dcgan.discriminator.parameters(), lr=0.0002, betas=(0.5, 0.999))

# Training loop
for epoch in range(num_epochs):
    for i, data in enumerate(dataloader):
        real_images = data[0]

        # Training the discriminator
        optimizer_D.zero_grad()
        output_real = dcgan.discriminator(real_images)
        loss_D_real = criterion(output_real, real_labels)

        noise = torch.randn(batch_size, 100, 1, 1)
        fake_images = dcgan.generator(noise)
        output_fake = dcgan.discriminator(fake_images.detach())
        loss_D_fake = criterion(output_fake, fake_labels)

        loss_D = loss_D_real + loss_D_fake
        loss_D.backward()
        optimizer_D.step()

        # Training the generator
        optimizer_G.zero_grad()
        output_fake = dcgan.discriminator(fake_images)
        loss_G = criterion(output_fake, real_labels)
        loss_G.backward()
        optimizer_G.step()

    print(f"Epoch_{epoch}/{num_epochs} Loss_D: {loss_D.item()} Loss_G: {loss_G.item()}")

```

Explanation: - The training routine uses a standard adversarial process for DCGAN, where the discriminator and generator are trained alternately. - The optimizer_G and optimizer_D are used to optimize the generator and discriminator, respectively. - The loss functions for both models (e.g., Binary Cross-Entropy) are computed and used to back-propagate gradients.

3) *WaveGAN Training Routine:* WaveGAN, designed for raw audio synthesis, follows a similar training structure but works on 1D audio data instead of images. The model uses 1D convolutions to generate waveforms.

Training Code Snippet for WaveGAN:

```

import torchaudio
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from models import WaveGAN

# Set parameters for the WaveGAN model
LATENT_DIM = 100
AUDIO_LENGTH = 16384

```

```

BATCH_SIZE = 16

# Load the audio dataset
audio_dataset = AudioDataset('./DATA/audio/')
dataloader = DataLoader(audio_dataset, batch_size=BATCH_SIZE, shuffle=True)

# Initialize WaveGAN model
wavegan = WaveGAN()

# Set up optimizers
optimizer_G = torch.optim.Adam(wavegan.generator.parameters(), lr=0.0001, betas=(0.5, 0.999))
optimizer_D = torch.optim.Adam(wavegan.discriminator.parameters(), lr=0.0001, betas=(0.5, 0.999))

# Training loop
for epoch in range(num_epochs):
    for i, real_audio in enumerate(dataloader):
        real_labels = torch.ones(batch_size, 1)
        fake_labels = torch.zeros(batch_size, 1)

        # Train discriminator
        optimizer_D.zero_grad()
        loss_D_real = criterion(wavegan.discriminator(real_audio), real_labels)
        fake_audio = wavegan.generator(torch.randn(batch_size, LATENT_DIM))
        loss_D_fake = criterion(wavegan.discriminator(fake_audio.detach()), fake_labels)
        loss_D = loss_D_real + loss_D_fake
        loss_D.backward()
        optimizer_D.step()

        # Train generator
        optimizer_G.zero_grad()
        loss_G = criterion(wavegan.discriminator(fake_audio), real_labels)
        loss_G.backward()
        optimizer_G.step()

    print(f"Epoch_{epoch}/{num_epochs} Loss_D: {loss_D.item()} Loss_G: {loss_G.item()}")

```

Explanation: - This code snippet demonstrates how to train the WaveGAN model with audio data. The audio is processed similarly to image data but uses 1D convolutions for generating waveforms. - The adversarial loss is computed and used to optimize both the discriminator and generator.

V. EXPERIMENTAL RESULTS

The results from training each model are saved in the RESULTS/ directory. Below is a summary of the results obtained for each model:

A. Text Generation (GPT-2)

The GPT-2 model was fine-tuned on a custom text corpus and evaluated using a test set of prompts. Generated text samples include:

```

RESULT 001
Prompt: What is North South University?
Generated on: 2025-07-30 02:05:19
Max length: 100
-----
Generated Answer:
What is North South University?
North South University (NSU) is a private research university located in Dhaka, Bangladesh. It was established on 12 January 2009 as a result of the Foundation for Promotion of Education and Research (FPER), which provided economic and administrative stability in Bangladesh. The university is widely regarded as one of the top private universities in the country, known for its academic excellence and modern campus facilities. The university is widely regarded as one of the top private universities

```


B. Image Generation (DCGAN, CGAN, CVAE)

For the image generation models, generated images were visually inspected for quality. Example outputs:

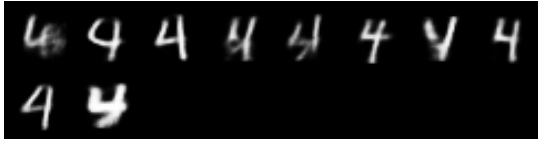


Fig. 2: Generated images using CVAE from the CIFAR-10 dataset

C. Audio Synthesis (WaveGAN)

Generated audio samples were evaluated qualitatively by listening to the output. Example audio files were saved as `generated_audio.wav` and could be played using an audio player.

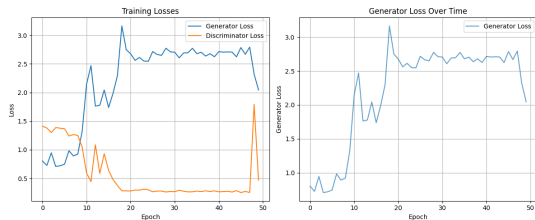


Fig. 3: Training Losses and Generator Loss Over Time of WaveGAN. You can listen to the audio at the following link: <https://colab.research.google.com/drive/12E3r3DdrFRSLvKpyXeYN1iOlswcJPoD?usp=sharing>

This concludes the description of the training workflows, followed by results for each model.

VI. COMPLEX ENGINEERING PROBLEMS (CEP)

A. P1: Depth of Knowledge (K3–K8)

The project involves several complex engineering problems, requiring deep expertise in multiple domains of artificial intelligence and software engineering. Below is a breakdown of the depth of knowledge required at various levels (K3–K8):

- **K3: Deep Learning (Transformer/GAN Architectures):** The project integrates state-of-the-art deep learning models, specifically transformer architectures (e.g., GPT-2) and Generative Adversarial Networks (GANs). Understanding the underlying theory behind these models, as well as their training procedures, is crucial for effective implementation and fine-tuning. This includes understanding the concepts of attention mechanisms in transformers and adversarial training in GANs.
- **K4: System Programming and CLI Design:** The system is built around a command-line interface (CLI), which required in-depth knowledge of system programming, including the design and development of an extensible and modular framework. This required integrating various models and datasets into a standardized execution pipeline that is user-friendly and highly configurable.

- **K5: Model Optimization and Fine-tuning:** Optimizing generative models for efficiency and accuracy is a central challenge of the project. Fine-tuning large pre-trained models like GPT-2 and GANs requires deep expertise in model architecture, hyperparameter tuning, and resource management, particularly when computational resources are limited. The project also involved training models from scratch for tasks like image and audio synthesis, which adds another layer of complexity.
- **K6: Tools (PyTorch, Hugging Face Transformers, TorchAudio):** To implement the models, the project utilizes several advanced AI tools and frameworks, including PyTorch for neural network development, Hugging Face's Transformers for fine-tuning GPT-2, and TorchAudio for handling audio data in WaveGAN. Mastery of these tools was essential for building the unified framework and ensuring compatibility across different model types and modalities.
- **K7: Ethical Considerations (Bias in Generated Data):** Generative models, particularly those trained on large datasets, can inadvertently reproduce biases present in the training data. It was crucial to consider these ethical concerns throughout the project, ensuring that the generated outputs are not perpetuating harmful stereotypes or unfair representations, particularly in sensitive applications like text generation or image synthesis.
- **K8: Research Alignment with Generative Modeling Studies:** The project aligns with contemporary research on generative modeling, particularly in the context of multimodal data. Understanding the research literature on the latest advancements in GANs, VAEs, transformers, and their cross-domain applications was crucial for selecting the right models and approaches. Moreover, aligning the project with current research practices ensured its relevance and potential impact in the field of AI.

B. P3: Depth of Analysis Required

The project required a substantial depth of analysis to evaluate and select the optimal combinations of architectures and strategies for each modality (text, image, and audio). The following aspects were analyzed in detail:

- **Model Comparison:** The project involved comparing several deep learning models—GPT-2, DCGAN, CGAN, CVAE, and WaveGAN—across different tasks. Each model has its strengths and weaknesses, and the goal was to select the most suitable model for the specific dataset and task at hand. For instance, GPT-2 was chosen for text generation due to its proficiency in natural language understanding and generation, while DCGAN was preferred for image generation due to its ability to produce high-quality images using unsupervised learning.
- **Fidelity vs. Latency Trade-Off:** Generative models often come with a trade-off between fidelity (quality of generated output) and latency (speed of generation). In the case of GPT-2, generating high-quality text at scale

requires significant computational resources, which directly impacts latency. Similarly, for GAN-based models like DCGAN and CGAN, generating high-quality images can be computationally expensive, particularly with large datasets. A detailed analysis was performed to optimize the models for both high-quality output and efficient training times.

- **Dataset Suitability:** Selecting appropriate datasets for each model was another crucial part of the analysis. For GPT-2, a large corpus of text data was essential for training. For image models like DCGAN and CGAN, carefully curated image datasets (e.g., CIFAR-10) were chosen to ensure that the generated outputs were realistic and meaningful. For WaveGAN, raw audio data was processed and converted into suitable formats to train the model effectively.
- **Hyperparameter Tuning:** Hyperparameter tuning was an essential part of the model optimization process. The project required careful selection of learning rates, batch sizes, and the number of epochs for each model to ensure the best possible performance. Additionally, the architecture of each model, such as the number of layers or the size of latent spaces, was adjusted for optimal results. For instance, larger latent vectors in GANs typically produce higher-quality outputs but at the cost of longer training times.
- **Evaluation Metrics:** The project evaluated the performance of each model using appropriate evaluation metrics. For GPT-2, text quality was assessed based on how coherent and contextually relevant the generated text was. For GANs and CVAEs, image quality was assessed through visual inspection, and for WaveGAN, audio quality was measured by perceptual audio scores. These metrics were essential for analyzing the trade-offs between model complexity, output quality, and training efficiency.

C. P4: Familiarity of Issues

The project requires familiarity with various technical issues related to both software and hardware integration. These challenges arise from the diverse generative models, their specific datasets, and the deployment on different computing platforms. Key issues include:

- **Generative Model Integration:** Combining different deep learning architectures, such as GPT-2, DCGAN, CGAN, CVAE, and WaveGAN, requires an understanding of model-specific training procedures, dataset compatibility, and hyperparameter tuning. Each model has unique requirements, such as text tokenization for GPT-2 and data augmentation for image generation models.
- **Cross-Modal Experimentation:** Ensuring that the framework supports experimentation across text, image, and audio modalities involves understanding how data from these different domains are processed, stored, and fed into the respective models. Data preprocessing steps

like tokenization for text or normalization for images must be carefully designed.

- **Dataset Compatibility:** Managing datasets for text (e.g., .txt, .docx, .pdf), images (e.g., CIFAR-10, CelebA), and audio (e.g., ESC-50) requires familiarity with various data formats and preprocessing pipelines. This includes handling text corpus formatting, image transformations, and audio waveform manipulation.
- **Hardware Limitations:** Generative models, particularly large ones like GPT-2, demand significant computational power. The limitations of available hardware (such as GPU memory constraints) require careful management of batch sizes, model checkpoints, and training durations to prevent system overloads and optimize performance.
- **CLI Usage and Scripting:** Familiarity with command-line interface (CLI) tools is essential for the usability of the framework. The models are managed through bash scripts and Python code, requiring knowledge of both bash scripting and Python programming to modify, extend, or troubleshoot the workflow.

VII. COMPLEX ENGINEERING ACTIVITIES (CEA)

A. A1: Range of Resources

The development of this unified generative AI CLI framework required substantial human and computational resources. The human resources involved include skilled developers responsible for coding, model optimization, and system integration. Hardware resources primarily consist of GPUs to handle the computational demands of training large models like GPT-2, DCGAN, and WaveGAN. Additionally, software libraries like `transformers`, `torchvision`, and `torchaudio` were critical for implementing and fine-tuning the models. Datasets such as CIFAR-10 (for images), Flowers (for conditional image generation), and various text corpora were used to train and evaluate the models.

B. A3: Level of Innovation

The integration of various generative models (GPT-2, DCGAN, CGAN, CVAE, and WaveGAN) into a unified CLI-based interface represents a significant innovation in the field of generative AI. While many of these models are available in the open-source community, they are often isolated, with separate codebases, dependencies, and workflows. By unifying them into a single framework with a standardized command-line interface, this project provides a more streamlined and efficient way for users to experiment with and deploy generative models. This approach is particularly beneficial for researchers, developers, and educators who want to explore multiple models across different domains without the need for extensive reconfiguration or specialized knowledge.

VIII. CONCLUSION AND FUTURE WORK

A. Summary

This project successfully integrated multiple generative models (GPT-2, DCGAN, CGAN, CVAE, and WaveGAN) into a unified CLI framework. The CLI tool simplifies the process

of training, evaluating, and generating outputs using different generative models for text, image, and audio modalities. The framework provides a modular, flexible interface, allowing users to quickly experiment with various datasets and models.

B. Limitations

While the unified CLI framework significantly reduces complexity, there are still some limitations. Training large models specially the Image generating models(CGAN, DCGAN, CVAE etc) requires significant computational resources and may result in long training times. Additionally, fine-tuning the models for domain-specific tasks may require additional hyperparameter adjustments. That is the reason why we couldn't use more advance models which gives generated text, images or audio based on a single prompt. The models which we worked on, although modular, are tightly coupled with specific dataset formats, meaning users must pre-process and organize their data accordingly.

C. Benefits and Educational Value

The unified CLI framework developed in this project provides a comprehensive platform for exploring generative AI models across multiple modalities, including text, images, and audio. By integrating well-known models like GPT-2, DCGAN, CGAN, CVAE, and WaveGAN, this project offers a streamlined and efficient way for both beginners and advanced users to interact with complex AI technologies.

D. For Beginners

- **Simplified Workflow:** The command-line interface abstracts the complexities of setting up and configuring different models, making it easier for beginners to get started with generative AI without needing deep knowledge of model internals. The provided shell scripts and Python code enable seamless execution, training, and evaluation of models.
- **Hands-On Learning:** By using this framework, beginners can experiment with popular generative models and observe the direct impact of hyperparameters and dataset choices. The clear structure of the framework promotes a deeper understanding of how different models work in the context of text, image, and audio generation.
- **Exposure to Multiple Modalities:** Beginners can learn how text generation (GPT-2), image generation (DCGAN, CGAN, CVAE), and audio synthesis (WaveGAN) differ in terms of architecture, data handling, and training procedures. This exposure provides a broader perspective on how generative AI models can be applied across various domains.
- **Educational Examples:** The code snippets, usage examples, and results provided in the framework serve as practical guides for learners to follow. They can modify these examples to better understand the nuances of training models and generating outputs, which is crucial for hands-on learning.

- **Modular Design for Easy Customization:** The modular architecture of the project allows beginners to dive deeper into specific parts of the pipeline, such as the model loading process, dataset preparation, or result generation. This flexibility fosters experimentation and customization, which is essential for understanding the intricacies of generative AI.
- **Access to Pre-trained Models:** Beginners can start with pre-trained models, fine-tune them on custom datasets, and generate outputs. This eliminates the need for extensive computational resources initially, allowing them to focus on learning the underlying principles of generative modeling without the challenges of training large models from scratch.

E. For Researchers and Developers

- **Unified Platform:** Researchers and developers can quickly deploy generative models for experiments without worrying about model-specific configurations or environments. The unified CLI framework ensures that different models (GPT-2, GANs, and VAEs) can be run and compared in a consistent manner.
- **Rapid Prototyping:** The modular design supports rapid experimentation with new datasets, hyperparameters, and even new models. Developers can integrate new architectures easily and compare their performance against existing models using a standardized evaluation setup.
- **Reproducibility:** The CLI setup promotes reproducibility by automating model training, evaluation, and result generation. This is particularly beneficial for research purposes, as it ensures that experiments can be easily replicated and verified by other researchers in the field.
- **Cross-Domain Applications:** This framework enables the study of generative models across different domains—text, image, and audio. This cross-modality capability helps developers understand how similar principles apply to diverse applications, opening the door for innovative multi-modal applications like text-to-image generation or interactive audio-visual systems.
- **Resource Efficiency:** The framework also facilitates resource-efficient experimentation by abstracting much of the overhead involved in training generative models. By focusing on the most important aspects of model development and evaluation, the system helps developers use computational resources more effectively.

F. Overall Impact

The framework is designed to democratize access to generative AI by lowering the barriers to entry for beginners while providing powerful tools for advanced users and researchers. With its modular design, pre-configured models, and easy-to-use CLI, it offers an accessible yet robust environment for anyone interested in learning about or experimenting with generative deep learning techniques. This project not only enables better understanding and exploration of generative models but also contributes to the broader goal of making AI

tools more accessible for educational purposes and practical use cases.

REFERENCES

- [1] Radford, A. et al., "Language Models are Unsupervised Multitask Learners", OpenAI, 2019.
- [2] Goodfellow, I. et al., "Generative Adversarial Nets", NeurIPS, 2014.
- [3] Donahue, C. et al., "Adversarial Audio Synthesis", ICLR, 2019.
- [4] Kingma, D. P., and Welling, M., "Auto-Encoding Variational Bayes", ICLR, 2014.
- [5] Radford, A., Narasimhan, K., Salimans, T., Sutskever, I., "Improving Language Understanding by Generative Pre-Training", 2018. [Online]. Available: <https://openai.com/research/language-unsupervised>
- [6] Huggingface, "Transformers: State-of-the-art Natural Language Processing for PyTorch and TensorFlow 2.0", [Online]. Available: <https://huggingface.co/transformers/>
- [7] Isola, P., Zhu, J. Y., Zhou, T., and Efros, A. A., "Image-to-Image Translation with Conditional Adversarial Networks", CVPR, 2017.
- [8] Donahue, J., and Simonyan, K., "Large Scale GAN Training for High Fidelity Natural Image Synthesis", NeurIPS, 2019.
- [9] Donahue, C., "WaveGAN: A Generative Adversarial Network for Raw Audio", ICLR, 2019. [Online]. Available: <https://arxiv.org/abs/1909.08007>
- [10] Paszke, A., et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library", NeurIPS, 2019.
- [11] "Torchaudio Documentation", [Online]. Available: <https://pytorch.org/audio/stable/index.html>